Distributed Cluster Monitoring System

Final Project Report

Project Plan Report

Team # 1

Smriti Reddy Uravakonda, Darsh Pandya

1 Project Goal

The goal of this project was to design and implement a fault-tolerant, distributed cluster monitoring system composed of replicated manager nodes and multiple workers. Manager nodes coordinate using a lightweight RAFT-like protocol to elect a leader, while workers periodically send heartbeats to the active leader. The system must detect failures, maintain cluster state reliably, and support automatic recovery after restarts or manager crashes. We have implemented a system which shows strong consistency and high availability to keep track of the worker nodes.

2 System Design and Assumptions

2.1 High-Level Architecture

The system consists of:

- **N manager nodes** implementing leader election, log replication, and state recovery.
- **N worker nodes** sending heartbeats at regular intervals.
- **A persistent JSON-based state store** that survives manager crashes.
- **A leader discovery service** for workers to find the currently active leader.

2.2 Key Assumptions

- At most one leader is active at any time in a healthy cluster.
- Network connectivity between managers may be unreliable, but eventually consistent.
- Workers communicate only with the current leader.
- A majority (quorum) of managers must be alive for leader election to succeed.
- Crashed managers may restart with stale state; the system must still converge.

2.3 Manager Responsibilities

- Participate in leader election (RAFT).
- Manage a worker registry (IDs, last-seen timestamps, status).
- Accept and process heartbeat messages from workers.

- Persist cluster state periodically for crash recovery.
- Publish a discovery port for workers when acting as leader.

2.4 Worker Responsibilities

- Continuously detect and connect to the current manager leader.
- Send periodic heartbeats.
- Retry on failure until the leader becomes available.
- Re-register automatically after leader failover or restart.

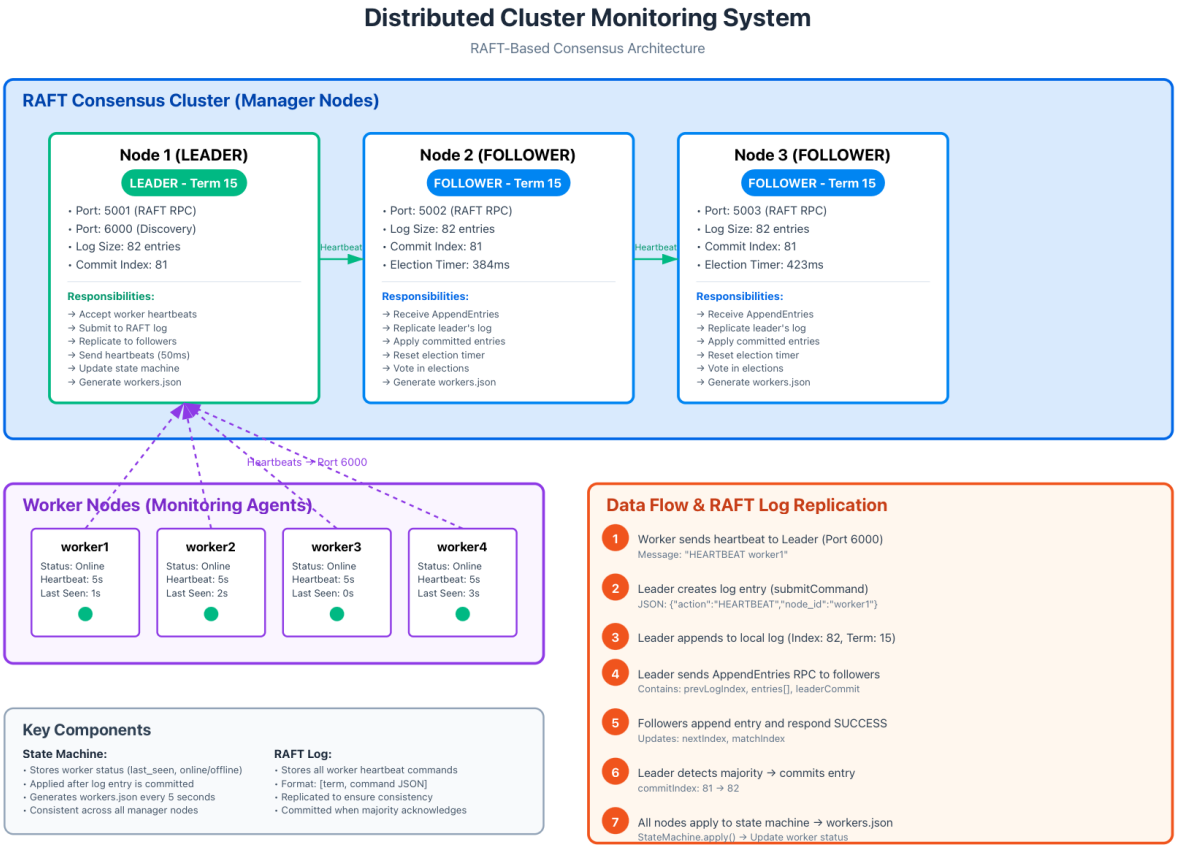

Figure 1: Design and Data Flow of 3 Node System.

3 What Was Achieved

3.1 Core Functionality

The following components were fully implemented:

- Complete RAFT like-protocol leader election among manager nodes.
- A working leader discovery service running at a fixed port.
- Worker-to-leader heartbeat protocol with ACK responses.
- In-memory log replication between manager nodes (logs are not persisted to disk).
- Failure detection using timeout-based logic.
- Persistent state storage using a JSON file.
- Automatic recovery of worker registry after manager restart.
- Quorum-aware behavior: election occurs only if a majority is online.

3.2 Fault Tolerance

- If the leader fails, a new leader is elected automatically.
- Workers automatically reconnects to the new leader at the leader discovery port when the previous leader crashes.
- Persistent storage ensures managers resume with correct state.
- Strong consistency is guaranteed across manager state machines.

4 What Was Not Achieved

We implemented all the goals we had set at the start of the project. However a few things which we could have further improved upon is:

- `Workers.json` may briefly continue updating after loss of quorum because the last leader continues running its local background thread.
- No formal test harness for network partitions or clock skews.

5 Evaluation of the System

5.1 Strengths

- Robust leader election, even under repeated failures.
- Workers reliably reconnect regardless of failures or restarts.
- Persistent state ensures clean recovery after node restarts.
- Heartbeat and timeout logic provides accurate failure detection.
- Architecture can scale to many workers with minimal overhead.

5.2 Weaknesses

- Quorum-loss handling stops new updates but requires additional cleanup to fully halt background threads.
- Workers.json may temporarily reflect outdated information (upto 5 seconds) until heartbeat cycles stabilize.
- Fixed Buffer sizes in RPC layer can cause JSON parsing errors.
- No persistent storage for RAFT logs: Log entries exist only in memory. Node crashes result in loss of historical log data, though the current worker registry state is persisted to workers.json and recovers from disk.

5.3 Observed Behaviors

- Elections converge quickly (under 2–3 seconds) in most runs.
- Workers send regular heartbeats and are correctly marked offline if missing.
- After restoring quorum, workers.json resumes updating as expected.

5.4 Automated Test Suite Summary

To validate the correctness, safety, and fault tolerance of the distributed monitoring system, an automated test suite (`run_tests_auto.sh`) was developed. The script executes manager and worker processes, injects failures, and validates cluster behavior under a variety of scenarios. The following summarizes the major categories of tests and their measured outcomes:

- **Correctness Tests**
  - Successful leader election was observed, with `node1` initially elected as the leader.
  - Workers successfully registered with the cluster; the system detected and tracked `worker1` and `worker2` within seconds.
  - State machine application was validated by comparing the last 10 log entries across all nodes. The leader and both followers applied **2 identical log entries**, confirming consistency.
  - Log replication correctness was confirmed:

    - Leader entries applied: 2
    - Node2 entries applied: 2
    - Node3 entries applied: 2

    The logs remained fully synchronized, indicating correct operation of the project-specific replication mechanism.

- **Concurrent Safety Tests**
  - The system handled **5 simultaneous worker connections**, scaling up from 2 to **7 total workers** without issue.
  - Log analysis showed **no race conditions**, deadlocks, or data corruption during concurrent updates.
  - Duplicate worker registrations were prevented successfully; all **7 workers were uniquely recorded**.
- **Fault Tolerance Tests**
  - When the active leader (`node1`) was terminated, the system automatically elected `node2` as the new leader, confirming expected failover behavior.
  - Quorum loss was correctly detected after terminating a second manager (`node3`), with Raft entering a non-operational state due to lack of majority.
  - After restoring `node1`, the system regained quorum and **resumed normal updates**, demonstrating robust recovery capabilities.
- **Performance Evaluation**
  - Heartbeat throughput over a 10-second measurement window recorded:
    - Total heartbeats processed: **14**
    - Average processing rate: **1.40 heartbeats/second**
  - The system remained responsive and stable under load as the number of workers scaled to 7.
  - Follower nodes applied the same state machine entries as the leader, further confirming correct replication behavior under load.

6 Achievements Compared to the Plan

The implemented system exceeds the basic monitoring requirement and includes:

- Fault-tolerant RAFT-like leader election (not required in original baseline).
- JSON-based persistent state storage.
- Automatic worker reconnection.
- A full automated test script for demonstrating failures.

7 Instructions for Setup / Run / Test

Khoury Linux Cluster Instructions

7.1 Login

```
ssh yourusername@linux-xx.khoury.northeastern.edu
```

7.2 Transfer Project

```
scp -r distributed-cluster-monitoring-system \
yourusername@linux-xx.khoury.northeastern.edu:~
```

7.3 Compile and Run

```
cd distributed-cluster-monitoring-system
make clean
make
```

7.4 Run Automated Test Script

```
chmod +x run_tests_auto.sh
./run_tests_auto.sh
```

In case you want to manually test anything:

7.5 Compiling the System

```
make clean
make
```

7.6 Running N Manager Nodes

Each manager requires:

- a unique ID
- a RAFT port
- a comma-separated list of peers

7.7 Start Managers

Example: Single Manager

```
./manager node1 5001 ""
(Yes, the quotes are important)
```

Example: Three Managers

```
./manager node1 5001 node2:5002,node3:5003
./manager node2 5002 node1:5001,node3:5003
./manager node3 5003 node1:5001,node2:5002
```

7.8 Start Workers

```
./worker_bin worker1
./worker_bin worker2
```

7.9 Detecting the Leader

Each manager writes logs to:

`node_nodeX.log`

The leader is the node whose log contains:

`Transition -> LEADER`

7.10 Testing Failure Recovery

- Kill the leader using `kill <PID>`.
- Observe that a new leader is elected.
- Restart the killed manager.
- Workers will automatically reconnect.

8 Individual Contributions

Smriti

- Implementing the Manager–Worker monitoring architecture.
- Implementing portions of the RPC layer and assisting with leader election stability improvements.
- Developing the persistent JSON-based state store and ensuring recovery from manager crashes.
- Designing and validating the worker auto-reconnection and leader discovery workflow.
- Creating and debugging the automated fault-tolerance test script.
- Running experiments on the Khoury Linux cluster and analyzing system behavior.
- Writing the majority of the final report and evaluation section.

Darsh

- Co-designing the distributed architecture and RAFT-based coordination logic.
- Integrating RAFT-style leader election and heartbeat-based worker health monitoring.
- Developing failure-handling logic and contributing to the timeout-based detection mechanism.
- Helping refine the persistent state model and validating correctness under restarts.
- Conducting extensive debugging during multi-node test runs and validating quorum behavior.
- Supporting the creation of instructions, documentation, and reproducible testing workflows.
- Performing performance measurements and helping evaluate system strengths and weaknesses.

Both of us collaborated closely throughout the project, reviewed each other's code, and participated in all design and debugging sessions. The final system reflects equal effort and shared understanding of the distributed monitoring architecture.

9 Conclusion

The project successfully delivers a distributed, fault-tolerant cluster monitoring system with automated leader election, persistent state, and resilient worker management. Despite omitting persistent log storage, the system demonstrates strong stability, scalability, and self-recovery properties under realistic failure scenarios.